NOEMA

The Trusted Execution Layer for Consequential Enterprise Action

As intelligence becomes abundant, the scarce infrastructure is trustworthy execution: the ability to turn human intent into correct, governed and auditable enterprise outcomes.

This is not a claim that models are unimportant. Models are the probabilistic intelligence that makes intent understandable. But understanding is not authority. A model can infer what someone probably meant; it cannot, by itself, establish whether that person is permitted to do it, whether the target is correct, whether the enterprise state is current, whether policy permits the action, whether an external system already accepted a request, or whether the result can be defended six months later. The hard part is not making software capable of taking an action. The hard part is making consequential action trustworthy enough to become authoritative enterprise reality.

This document is the canonical account of what Noema is: the bet, the problem, the solution, the two modes , the market, the competition, the moats, the proof we are building first, the benchmark , and the risks we hold honestly, the future. It is written to remove ambiguity, not to impress. Every external number is sourced inline and was verified against primary or major reporting at the time of writing. Where something is our judgment rather than a fact, we say so.

HOW TO READ THIS DOCUMENT — AND THE ONE DISTINCTION YOU MUST HOLD

Two things need to be unmistakable from the first page, because confusing them is the fastest way to misunderstand this company.

First: Noema is not the same thing as Noema's Proof of Concept. They are different in scope, in problem, in solution, in market, and in numbers.

- **Noema (the company)** is a horizontal trust-and-execution layer for *consequential software actions across every industry* — banking, insurance, telecom, healthcare, logistics, procurement, government, and any enterprise where a human or an AI expresses an intention that has real consequences. Its opportunity is measured across the whole enterprise economy, not one sector.
- **The Proof of Concept (POC)** is *one deliberately narrow, brutally hard workflow* — resolving failed and disputed payment transactions in Nigerian fintech. It is what we are building **first**, to prove the core engine under the most hostile conditions we could find. It is a thin vertical slice, not the company. *We are starting inside Nigerian fintech, on one narrow workflow, because it is one of the hardest places in the world to earn the right to move real money on someone's behalf. If it survives here, we believe it survives almost anywhere. That is the whole shape of the bet.*

Throughout this document: when we discuss the **opportunity, the problem, the solution, or the market**, we mean the company. Those numbers are never constrained to fintech or to Nigeria. When we discuss **what we are building in the coming weeks**, we mean the POC. The POC's numbers are the numbers of its specific wedge, and they are labelled as such. We never inflate the company by borrowing the POC's boundaries, and we never inflate the POC by borrowing the company's scope.

Second: we are not building Noema for enterprises to use internally on themselves. We are building Noema for enterprises to integrate into their own products and offer to their own customers. Noema is infrastructure an enterprise embeds so that *its* customers get the outcome they want, safely, without fighting an interface or waiting on a human. We sell to the enterprise; the enterprise's customer is who ultimately feels the benefit. Keep this in mind everywhere below — it shapes the product, the pricing, and the go-to-market.

PART ONE — THE BET

Noema in one sentence

Humans should express goals. Software should handle operational complexity.

Noema is the trusted execution layer that sits between any source of intent — a human, an AI agent, or another

system — and the enterprise systems that carry out consequential actions, and it guarantees that intent becomes a correct, authorised, verified, and auditable outcome, every time, or is safely refused.

Everything else in this document explains that sentence, defends it with evidence, and describes how we intend to earn the right to build it.

The belief behind Noema

People never wanted software. They wanted outcomes. The application was only ever the bridge to the outcome.

For fifty years, software worked one way. A human wanted something. Before a machine would do it, the human first had to translate that want into the machine's language — its screens, menus, forms, and exact sequence of taps. We called that translation "using software," and we built an entire civilisation of interfaces on top of it.

That translation was never a law of nature. It was a work a round for a limitation: machines could not understand what people meant, so people learned to think like machines. **That limitation has now broken.** For the first time, a machine can take a human intention, expressed in ordinary, messy, mixed language, and understand it well enough to act on it.

Most of the industry read this moment as *"the interface is going to get smarter."* We read it as something more fundamental:

Because understanding was never the whole job. A bank does not struggle because it cannot understand "send twenty thousand to Tunde." It struggles because, once it understands, it must still answer a chain of questions that have nothing to do with language: *Which Tunde? Which account? Is this person allowed to do this? Does this need a second approval? Is this consistent with what is actually true across our systems right now? What happens if the network drops halfway through? Can we prove, afterward, exactly why we did it?*

Those are not intelligence problems. They are trust, state, governance, and execution problems. And they get *harder*, not easier, the moment you let an autonomous system act on real consequences.

The bold bet, stated plainly

Humans should express goals. Software should handle operational complexity.

Over the next decade, the dominant way people interact with enterprise software will move from learning interfaces to expressing outcomes. A growing share of enterprise interactions — customer requests, internal operations, exceptions, cross-system tasks — will shift from manual forms and call centres to unstructured human expression, and increasingly to AI agents acting on people's behalf.

But **enterprise software cannot run on probabilistic guesses.** Today's language models and agent frameworks can increasingly understand natural language, reason over context, call tools and generate plans but are structurally unfit to be trusted with consequential execution on their own: they are probabilistic by design, they hallucinate, they cannot guarantee state consistency, they are vulnerable to confused-deputy attacks, and they cannot produce the deterministic, audited state changes that regulated enterprises require.

Our bold bet is that the winning enterprise infrastructure of the AI era will not be another model or another agent framework. It will be the deterministic trust-and-execution boundary that bridges human intent and enterprise systems — the layer that decides whether a consequential action should become real, does it safely, verifies it happened, and can prove why.

The fundamental equation of the company:

Noema enforces an absolute separation between **probabilistic operations** (interpreting messy, code-switched human intent) and **deterministic operations** (evaluating authorisation, enforcing policy, reconciling multi-system state, executing, and generating tamper-evident audit trails). Models propose. Deterministic systems decide and execute. That separation is the whole architecture, and it is the reason the system can be trusted with consequences.

Over the next decade, we expect five behaviors to become ordinary:

1. People will describe outcomes instead of navigating every > intermediate screen.
2. Personal and enterprise agents will increasingly act for users, but > their authority will remain bounded.
3. One task will routinely span several systems of record rather than a > single application.
4. The model layer will remain plural and fast-changing; enterprises > will not want the safety of critical workflows tied to one model > vendor.
5. Trust will migrate from "the user clicked the approved button" to > "the system can prove who intended what, what was

authorized, what > state existed, what policy applied, what happened and whether the > outcome was verified."

If the next layer of enterprise AI simply adds more interfaces and more agents, the enterprise inherits more complexity, not less. We are solving this because the useful endpoint of AI is not a better conversation. It is **more work completed correctly, with less human operational burden, without giving up trust.**

The evidence that this is real, not a hunch

We are not the only ones who see the gap. The most credible warning came from the analysts enterprises actually listen to.

Gartner estimates that up to $234 billion of enterprise application spending could be exposed to "agentic arbitrage" by 2030 as agents complete tasks across systems and reduce the need for users to interact with multiple application interfaces. Gartner explicitly describes a future in which agentic systems deliver outcomes directly and make software less visible to the user. This means enterprise software economics are beginning to move from interfaces and seats toward outcomes and execution.[1]

In June 2025, **Gartner predicted that over 40% of agentic AI projects will be cancelled by the end of 2027**, and was explicit that the cause is "escalating costs, lack of trust, governance, inadequate risk controls and clear return on investment" — not a failure of the models. This indicates the business does not fundamentally want more AI activity. It wants a result produced with acceptable cost, risk and accountability.[1]

A year later, a Forbes analysis put it bluntly: the coming cancellation wave is *"a management problem wearing a technology costume,"* and the agents that survive "won't be the ones running the largest models" — they'll be "the ones with a number attached to their job and a name on the override switch."[2]

Independent research is harsher still: **RAND found AI project failure rates exceed 80% — more than twice the rate of non-AI technology projects.**[3]

Deloitte's August 2026 survey gives the transition some scale: 74% of surveyed leaders expect nearly half of business processes to be redesigned or rebuilt around AI agents within four years; 61% expect most agents they use to be generally autonomous. Yet only 5% say their business processes are highly prepared for agents, only 15% report scaled orchestrated multi-agent adoption, 70% say they cannot yet trust and govern agents, and 67% say integration is too costly and complex. The gap between desire for autonomy and readiness for trustworthy execution is the space Noema is built for.[1]

Read those five facts together and the shape of the opportunity is exact. These are clearest validation we have found that this specific gap is real and industry-wide, not a story we invented to sound differentiated. None of these numbers, by themselves, prove Noema will win. What they prove is that the pain is real, it is large, it is measured by people with no reason to exaggerate it, and it is getting worse, not better, on its own.

The market is not short of intelligence — it is drowning in it. What is killing real deployments in real companies right now is the absence of a trustworthy way to let that intelligence *act*. **That is precisely the layer Noema is building.**

[1] Gartner, 25 June 2025. 2 R. J. Szczerba, Forbes, 7 July 2026. [3] RAND Corporation (Ryseff & Narayanan), 2025–2026.

The Paradigm Shift - Intent Computing

We use the term Intent Computing to describe the broader paradigm. Intent Computing is not simply natural-language software. It is software organized around what a person is trying to accomplish rather than the interface they have learned to navigate.

Intent Computing describes the shift from navigating software to stating outcomes. The human says what they want. The system determines the path.

The important distinction is:

Traditional computing

Human → learns software → navigates workflow → produces outcome

Intent computing

Human → expresses intent → system resolves meaning → system governs execution → outcome

That shift creates a new architectural requirement. The system cannot simply understand language and call tools. It must bridge the gap between probabilistic understanding and authoritative enterprise execution. The system still has to do the difficult work underneath. It must identify the right objects, reconcile conflicting records, respect authority, apply policy, handle ambiguity, execute across several systems, survive timeouts and partial failure, verify the real-world result, and preserve accountability. That is where Noema sits.

Noema is betting that this boundary becomes a foundational layer of enterprise computing in the same way identity, databases, networking, cloud infrastructure and payment rails became foundational layers. We do not need to own the interface through which people express intent. We need to become the infrastructure those interfaces rely on when intent can cause a real enterprise consequence.

Intent Computing is the paradigm; Trusted Enterprise Execution Infrastructure is the category Noema intends to build within it.

PART TWO — THE PROBLEM (THE REAL ONE, NOT THE POC's)

The problem Noema exists to solve

Enterprise software is excellent at storing records and exposing functions. It is much weaker at turning ambiguous human goals into safe, verified business outcomes across systems. The gap is widening because AI makes interpretation and tool use cheap, while the cost of being wrong remains high.

When a person or an AI expresses an intention that has real consequences, there is no reliable, governed, auditable layer that turns that intention into a correct action inside enterprise systems — and executes it safely, every single time, or refuses to.

Today, that job is done by humans. An employee reads the request, works out what it actually means, checks whether it is allowed, gathers context from multiple systems, decides, does it, and — if anyone ever asks — reconstructs why. **The enterprise has effectively built a manual execution engine out of people.** Multiply that across every consequential action in every enterprise and you have an enormous, invisible tax: slow, expensive, inconsistent, error-prone, and impossible to scale.

The problem has six structural layers, and each one is a reason the naive "just connect an LLM to the API" approach fails.

1. **Human intent is lossy.** People express goals ambiguously, in mixed language, with missing details. "Send the money to Tunde" is not executable — which Tunde, which account, from where, subject to what limit. Resolving that ambiguity *against real data* rather than guessing is the first hard problem.
2. **Enterprise state is fragmented.** The truth about a single event is scattered across systems that frequently disagree. For one payment: one system knows the debit, another the switch response, another settlement, another the beneficiary account, another the fraud status, another the support case, another the customer's identity and permissions. The enterprise has all the pieces. What it lacks is a reliable way to answer the cross-system question: *"What actually happened, and what should we do now?"*
3. **Authorisation is more than authentication.** Knowing *who* is asking is not the same as knowing whether *this specific action, on this specific resolved target, under the current real state, right now* should be allowed. Static role checks cannot answer that. It requires business-level authorisation evaluated against live reality.
4. **Execution is not the same as calling a tool.** A tool call can succeed at the API level and still be wrong — executed twice, executed with a wrong parameter, or reported as "success" while the real-world outcome never happened. Safe execution requires idempotency, verification against actual state, and recovery when things go wrong halfway.
5. **AI amplifies the consequence gap.** Handing an unsupervised probabilistic system the authority to act on real consequences does not shrink this problem — it multiplies it. A model that is 95% accurate is, at enterprise scale, a system that produces 50,000 wrong outcomes per million actions. In consequential work, that is not a rounding error. It is a catastrophe with a demo attached.
6. **Accountability is becoming a product requirement.** As automated systems take more consequential actions, "prove why the system did that" is becoming a hard requirement — from regulators, from compliance teams, from customers. Raw API logs and token traces do not meet that bar. A complete, tamper-evident causal record does.

Why this is worth a company — and why it becomes more urgent as AI improves

Here is the counter-intuitive core of the thesis: **better models make this problem bigger, not smaller.**

The better models get at understanding intent, the more enterprises will want to let them *act*. The more they act, the more consequential the actions become. And the more consequential the actions, the more unbearable it becomes to have no trustworthy layer governing whether those actions should happen and proving that they did. Perfect understanding does not answer "is this authorised, is it consistent with current state, what happens on partial failure, can we prove it." Those questions get harder as autonomy rises.

So this is not a problem that a smarter model eventually absorbs. It is a problem that a smarter model makes more valuable to

solve. Intelligence determines what *could* happen. Trusted execution determines whether it *safely should* — and whether it actually did.

This is not an internal-tools problem. It is an integration problem.

One point that must be crystal clear, because it shapes everything: **we are not building Noema for enterprises to run their own internal operations on themselves. We are building Noema for enterprises to embed into their own products and offer to their own customers.**

A bank integrates Noema so its customers can resolve a problem by simply saying what they need. An insurer integrates Noema so policyholders get claims outcomes without navigating a portal. A telecom integrates Noema so subscribers get provisioning issues fixed through natural expression. In every case, Noema is invisible infrastructure inside the enterprise's own branded experience, and the person who feels the benefit is the enterprise's customer. We sell to the enterprise. The enterprise's customer gets the outcome. That is the shape of the business.

PART THREE — THE SOLUTION

What Noema actually is

Noema is a **trusted execution layer** between any source of intent (a human, an agent, or another system) and any enterprise system that carries out consequential actions (a bank ledger, an ERP, a CRM, a healthcare record, a payments rail, a government registry).

Noema takes a probabilistic, messy, human expression of intent and decides — deterministically, with evidence, with an audit trail a regulator can stand behind — whether that intent should become a real, consequential action inside an enterprise's own systems.

In plain terms: a person says what they want, in their own words, in whatever language they naturally mix. Noema figures out exactly what they mean, what is true now, what the actor is allowed to do, what policy permits, checks whether it is allowed, does it — or safely refuses and explains why — and leaves behind a complete, provable record of what happened and why. The enterprise's own systems remain the system of record. Noema never becomes a shadow bank or a shadow database. It is the layer that decides, safely, whether an action should be allowed to happen at all.

Its job is everything that must be true *before* a consequential action touches a real system, and everything that must be verified *after*:

- **Understand** the true intent — resolving ambiguity against real enterprise data, never guessing.
- **Reconstruct state** — assemble one authoritative truth from systems that may disagree, be stale, or be incomplete.
- **Establish trust** — is the requester authorised for *this specific action on this specific target under the current real state*, does policy permit it, does it need another approval, is it unusual, is it consistent.
- **Plan** the execution — in what order, across which systems, with what to do if a step fails halfway.
- **Execute** deterministically and idempotently — never twice, never with an unverified parameter, only through governed, bounded capabilities.
- **Verify** the real outcome — not "the API returned 200," but "the thing the person actually needed genuinely happened."
- **Prove** it — a complete, reconstructable, tamper-evident record of who, what, why, under which policy, against what evidence.

And when something does not check out, Noema does not guess. **It stops, asks, escalates, or refuses.** The willingness to say "I am not sure, so I will not act" is not a weakness of the system. It is the entire product.

The complete Noema solution — the components

The full Noema platform is composed of the following systems. (The POC implements a focused subset of these — Section 19 makes that explicit.)

1. **Intent Compiler / Intent Engine.** Turns messy, code-switched human expression into a strictly-typed, validated **Canonical Intent Object** — the durable, structured representation of what was asked. This is the one place probabilistic inference is permitted, and its output is schema-validated at a hard boundary before anything downstream can touch it.

2. **Context Fabric / Context Engine.** Enriches the intent with everything needed to interpret it correctly for this specific requester, right now: identity, permissions, history, session, device, and the candidate resolution of ambiguous references ("which Tunde," "which order," "which subscription").

3. **State Reconciliation.** Reconstructs one authoritative truth from multiple enterprise systems that may disagree, be delayed, or be stale — classifying each fact by whether it is known, inferred, ambiguous, stale, contradictory, or unknown. "I don't know yet" is a first-class, valid system state.

4. **Trust Engine.** A deterministic decision system — not a model — that evaluates identity, authority, policy, risk, state consistency, and ambiguity, and returns exactly one enumerated outcome: allow, clarify, confirm, wait, escalate, reject, or unknown. Nothing consequential executes without its explicit clearance.

5. **Enterprise Action Graph / Outcome Graph.** The earned, executable map of how a specific enterprise's systems actually expose capabilities, behave, fail, and recover — and how an intent maps to the real actions and outcomes that satisfy it.

6. **Execution Controller.** Invokes only pre-registered, bounded capabilities. Idempotent by construction — never executes twice, never with an unverified parameter, never a prohibited action.

7. **Verification & Recovery.** Confirms the real-world outcome against expectation rather than assuming success, and safely recovers or escalates on partial failure, timeout, or unknown outcome — never a blind retry.

8. **Causal Audit & Explanation.** Produces a complete, tamper-evident, reconstructable record of every decision, and a plain-language explanation grounded only in verified facts.

9. **Operational Memory.** Accumulates what the platform learns — patterns, failures, resolutions — so every interaction and every integration makes the next one better.

Two modes of Noema — and why we build it this way

This is one of the most important strategic decisions in the company, so we state it directly. Noema is designed to be deployed in **two modes, from the same core.**

Mode 1 — the full-stack execution platform. Noema owns the whole path: intent arrives through any interface, Noema understands it, reconstructs state, establishes trust, plans, executes against the enterprise's systems, verifies, and proves. This is the mode for an enterprise that wants the entire capability embedded in its product and does not already have an agent stack. Noema owns the entire path: human expression comes in through voice, text, or WhatsApp, and Noema runs the full loop — understanding, trust, execution, verification — all the way to the enterprise's own systems. This is the version we are proving first, because it produces the clearest, most complete demonstration of the whole value chain in one motion.

Mode 2 — the independent trust boundary for consequential actions. Here the enterprise already has AI agents — their own, or built on a framework, or on a hyperscaler's agent platform. Those agents *propose* actions. Noema sits at the boundary and decides whether each proposed consequential action is actually allowed to become real — evaluating it against real enterprise state, policy, authority, and business context, then executing safely and proving it. In this mode, Noema is the mandatory, trusted checkpoint every consequential action must pass through, no matter which agent or model produced it, and the single place a compliance officer can look to answer "who did what, under which policy." For an enterprise that has already invested in an agent framework or a governance layer like AWS Bedrock AgentCore or Harness, Noema does not compete with that investment or ask them to rip it out. It sits downstream,

Why both? Because they are the same core capability pointed at two different market realities, and together they cover the entire future. If an enterprise has no agents yet, we are their execution platform. If they already have a dozen agents from a dozen teams, we are the single governed boundary that makes all of them safe. **One mode wins the greenfield. The other wins the moment agents proliferate — which is exactly the moment the trust problem becomes unbearable.** We do not have to bet on which future arrives. We are positioned for both, from one architecture.

Interface-agnostic and model-agnostic — by design, not by accident

Two properties make this a durable company rather than a feature, and they are deliberate architectural commitments.

Interface-agnostic. We treat every interface as interchangeable and temporary. Voice is where we start — it requires no literacy, works on any device with a microphone, and is the fastest way to prove the thesis with real financial stakes. But the underlying engine never assumes voice. The same Intent Object that a spoken sentence produces is produced identically from text, chat, WhatsApp, wearables, smart devices, an API call from another system, an instruction from an AI agent, or an interface that does not exist yet — Noema does not care how intent arrives. It cares only about what happens *after* intent is understood. We are not betting on interface because interfaces churn constantly, and every company that ties itself to one interface dies when the interface changes. Infrastructure that sits *beneath* every interface compounds instead of churning.

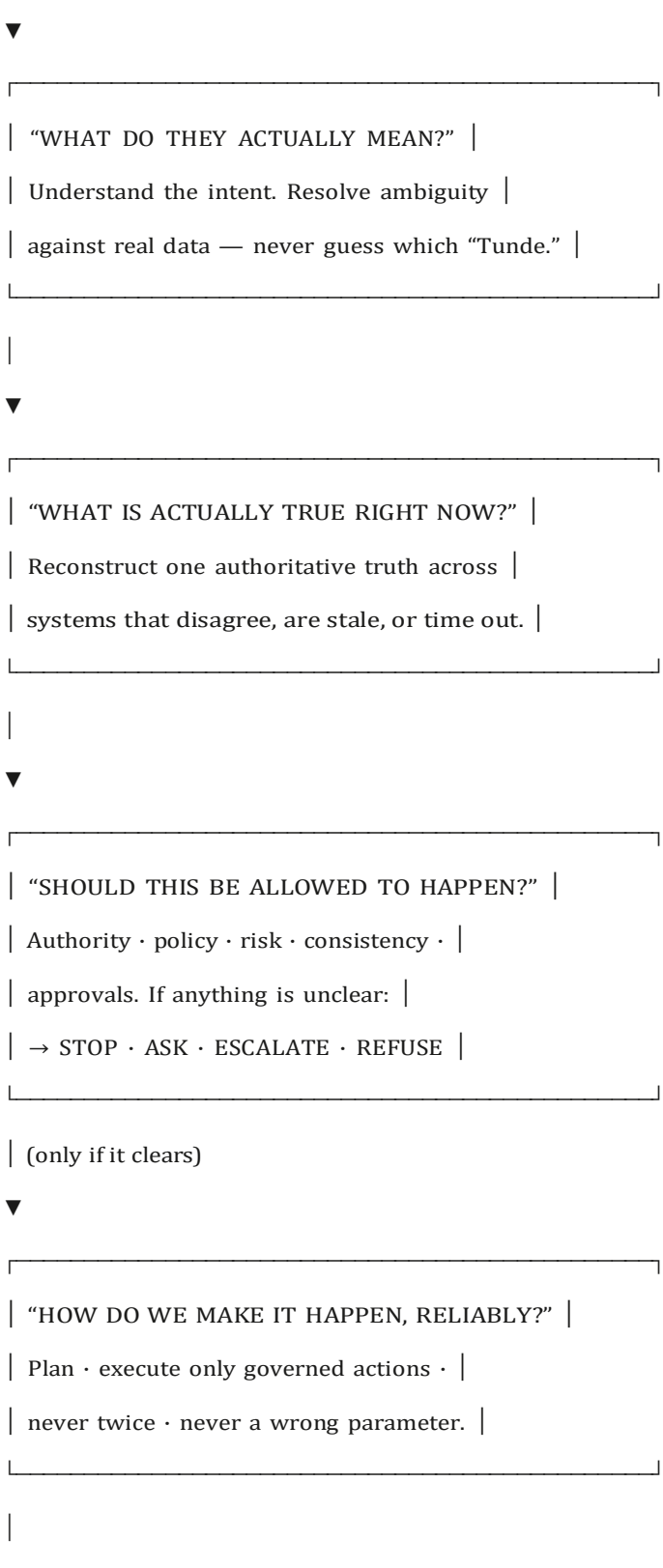
Model-agnostic. The same discipline applies to models. Foundation models are commoditizing quickly and will keep getting cheaper and more capable — that is a tailwind for us, not a threat, because cheaper intelligence means more volume flowing through the trust and execution layer we own. We do not compete with OpenAI, Anthropic, or Google, and we do not depend on any single one of them. Models propose; Noema decides and executes. When the next, better, cheaper model arrives, we get stronger, not obsolete — because the hard, durable, defensible work was never the model. It was everything between the model's proposal and the real-world consequence.

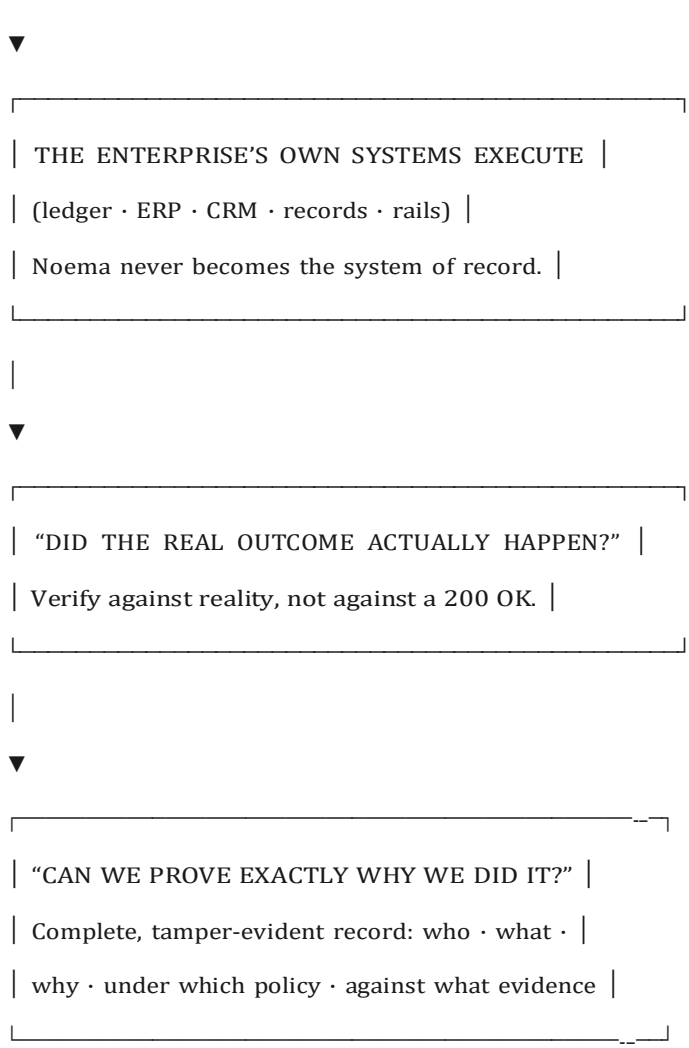
What the experience looks like when it works

Here is the whole system, drawn the way a business leader should understand it — not as engineering boxes, but as the questions each layer answers on the enterprise's behalf, so that the enterprise's *customer* simply gets the outcome.

A HUMAN OR AN AI EXPRESSES AN INTENT

(voice · text · chat · WhatsApp · API · another agent)





The experience should feel almost simple, because the complexity is underneath.

With Noema

Person or agent -> expresses desired outcome -> Noema resolves meaning + authority + state + policy -> Noema executes through existing systems -> Noema verifies -> outcome + explanation + audit

Instead of

Person -> finds the right application -> learns its language -> navigates screens -> discovers missing context -> opens another system -> asks support -> waits for a specialist -> specialist checks multiple systems -> applies policy -> executes -> manually confirms -> explains

The user should not have to know which system owns the customer record, which service owns policy, which workflow needs an approval or which API creates a case. The enterprise should not have to trust the model to improvise those rules.s

Every box is a job an enterprise currently pays humans to do, or dangerously hopes an unsupervised agent will do correctly. Noema does all of them, deterministically, at machine scale, with a receipt — so the enterprise's customer just gets the answer.

What this looks like across industries — because it is not a fintech product

The same core capability applies anywhere a human or an agent expresses a consequential intent. Four concrete examples, deliberately outside payments, to make the horizontal nature unmistakable:

Procurement. A manager says: *"Order the standard laptops for the four engineers starting Monday, within the approved onboarding budget."* Noema resolves which four employees are in scope, which device standard applies to their roles, whether the manager is an authorised requester, the current budget, approved suppliers, purchasing policy, and existing orders. If one engineer already has an order, it does not duplicate it. If the budget is insufficient, it explains the exact constraint and routes the right approval rather than "trying its best."

Customer operations. A customer says: *"I cancelled last week but you charged me again. Fix it."* Noema identifies the subscription, verifies the cancellation state and billing timeline, determines refund eligibility under policy and jurisdiction, distinguishes a pending authorisation from a settled charge, executes only the approved remedy, verifies the ledger state, and explains precisely.

Employee operations. An employee says: *"My manager changed. Give the new manager the approvals she needs, but do not change my project access."* Noema treats this as far more than an identity API call — determining the requestor's authority, which approvals the organisational change implies, which access is explicitly excluded, whether segregation-of-duties rules apply, and verifying propagation across authoritative systems.

An existing enterprise agent (Mode 2). A service agent the enterprise built proposes: *"Issue a £480 goodwill credit to this account."* Noema does not own the conversation. It receives the proposed action plus its principal and context, resolves the account and policy, verifies authority and thresholds, checks for duplicates and recent credits, determines whether extra approval is needed, executes through a bounded capability if allowed, verifies the result, and returns a signed decision-and-execution record.

The principle across all four: **natural expression at the edge; deterministic trust at the point of consequence.**

Where Noema can be used — horizontal by design, selective by go-to-market

Noema is horizontal in its **infrastructure** and deliberately selective in its **go-to-market**. We never confuse "can apply across industries" with "should sell to everyone at once." The table below is not a list of products we are shipping — it is evidence that the same underlying execution contract recurs everywhere consequential work happens.

Domain	The human outcome	What makes it consequential
Finance operations	"Resolve this duplicate vendor payment and close the exception."	Money, approvals, ledger state, segregation of duties
Procurement	"Buy the approved equipment for these hires."	Budget, supplier policy, duplicates, approvals
Customer operations	"Correct this billing error and tell the customer what changed."	Entitlements, refunds/credits, policy, identity
HR / people operations	"Update this employee's role and the access that follows from it."	Identity, employment state, access, compliance
Insurance	"Resolve this claim based on the policy and evidence."	Eligibility, evidence, payment authority, regulation
Logistics	"Reroute this shipment without breaking the delivery commitment."	Inventory, contractual SLA, carrier state, cost
Healthcare administration	"Reschedule this procedure and preserve the required authorization."	Sensitive data, eligibility, capacity, safety-adjacent constraints
IT operations	"Restore this service without executing an unsafe change."	Production state, change policy, rollback, availability
Public services	"Update this case after verifying eligibility and required evidence."	Rights, identity, policy, auditability

The product thesis is not that one generic model prompt can solve all of these. It is that the **execution contract** — intent, state, authority, policy, bounded action, verification, and provenance — generalises, while each domain supplies its own semantics, capabilities, and policies. That is precisely what makes Noema infrastructure rather than an application: the contract is universal; the domain content is configuration.

Tangible business value: why an enterprise buys Noema

Enterprises do not buy a new infrastructure category because the architecture is elegant. They buy when it moves a budget line, a risk line or a growth line. Noema must deliver value in six measurable ways.

Lower cost to serve. When a workflow requires a human to identify context, copy information between systems, interpret policy, create a case and explain the result, cost grows with volume. Noema can automate the repeatable reasoning and execution path while escalating only the cases that genuinely require judgment or authorization.

Measures: manual touches per case, handling time, cases per operator, support contacts per outcome, escalation rate, cost per resolved outcome dramatically.

Faster time to outcome. The enterprise often has the data and capabilities needed to resolve a request but not a single mechanism to assemble them. Noema compresses handoffs and system navigation.

Measures: median/P95 resolution time, queue age, time in each handoff, SLA compliance, first-contact resolution.

Safer autonomy. The business value of autonomy is not maximum autonomous action. It is maximum correct autonomous resolution inside an acceptable safety envelope. Noema should allow enterprises to move more consequential work out of manual queues without granting an agent unbounded authority.

Measures: autonomous correct resolution rate, unsafe action rate, false-block rate, intervention rate, policy violations, unauthorized disclosures.

Fewer integration projects per interface. As enterprises add assistants, departmental agents, chat surfaces and devices, rebuilding business rules and execution safety inside every interface becomes expensive. A common execution layer lets the enterprise reuse trusted capabilities beneath many entry points.

Deloitte reports 67% of surveyed leaders see integration as too costly and complex for agentic transformation; MuleSoft reports IT teams spend 39% of their time on custom integration work.

Measures: integration lead time, capability reuse across surfaces, maintenance hours, duplicated business logic retired.

Better customer and employee experience. A user who can accomplish an outcome without learning a complex product has a shorter path to value. This can improve onboarding, accessibility, completion and retention while reducing dependence on training and support.

Measures: task completion, abandonment, repeat contacts, CSAT/NPS where appropriate, onboarding time, support deflection with verified resolution rather than answer-only deflection.

Stronger audit and regulatory posture. Noema's causal audit is intended to answer the business question, not merely reconstruct a model trace. This reduces the time and ambiguity involved in incident review, complaint reconstruction and control testing.

Measures: audit reconstruction time, evidence completeness, policy-version traceability, incident mean time to understand, regulatory complaint aging.

The buying proposition

The enterprise should buy Noema when the following inequality is true:

Value of faster + safer + cheaper outcomes across a painful workflow > integration + platform + change-management cost.

If we cannot prove that inequality in production, the company thesis is not commercially validated.

Why this becomes possible now

Three forces arrived together. Any one alone would be insufficient.

Foundation models can now propose structured actions reliably. The breakthrough of the last two years was not raw intelligence — it was *dependable structure*: models that reliably return typed, validated output and hold a multi-step plan. That is the raw material a trust layer needs. Two years ago it hallucinated too much to build on. Now it does not.

Enterprise functions are increasingly exposed as APIs and tools. A decade ago, even a perfectly understood intent often had nowhere to go — the action lived behind a screen, not an endpoint. Today the overwhelming majority of enterprises are API-first, and standards like MCP are making enterprise capabilities addressable to AI. Intent finally has somewhere to go.

Identity, policy, and durable-execution infrastructure is becoming agent-aware, and the consequence of getting it wrong is now visible and urgent. As agents move from answering to acting, every enterprise is asking the same question at once: *who governs what the AI is actually allowed to do?* The market has started building the narrow, technical half of that answer. Nobody has built the whole answer. That is the opening.

Durable execution is a solved-enough substrate. Platforms such as Temporal have made durable workflow state, retries and long-running execution accessible infrastructure. Noema can use or interoperate with mature durability primitives instead of pretending retry semantics are a unique invention.

PART FOUR — THE SCALE OF THE OPPORTUNITY (THE COMPANY, NOT THE POC)

This is a computing-layer opportunity, not an industry opportunity

We are careful and honest here, because it is the most abused section of any company document. We will not inflate, and we

will not fabricate a TAM by assuming a capture rate. We show the real, sourced numbers and say clearly what they do and do not prove.

The direct market — AI agents that take real actions in enterprises — is already large and compounding at an extraordinary rate. Independent research houses, using different methods, converge on the same story:

The direct market — AI agents that take real actions in enterprises — is already large and compounding at an extraordinary rate. Independent research houses, using different methods, converge on the same story:

Source (2025-2026)	Market	Size today	Size ~2030	CAGR
MarketsandMarkets	AI Agents	$7.84B (2025)	**$52.62B (2030)**	46.3%
MarketsandMarkets	Enterprise Agentic AI	$6.76B (2025)	$46.04B (2030)	47%
Grand View Research	Enterprise Agentic AI	$2.6B (2024)	$24.5B (2030)	46.2%
Precedence Research	AI Agents (broad)	$7.92B (2025)	**$236B (2034)**	—

Sources: MarketsandMarkets, Grand View Research, Precedence Research, 2025–2026.

More directly relevant, Gartner estimates that up to **$234 billion by 2030** of enterprise application spending could be exposed to agentic arbitrage - roughly 20% of enterprise application SaaS spend - as agents perform tasks across systems and reduce direct interaction with traditional interfaces.

Every one of these numbers describes the market for AI *taking action* — and every dollar of it runs into the wall Gartner described: the 40% cancelled for lack of trust, governance, and ROI. **Noema is not one vendor inside that market. Noema is the layer that determines whether that market's spending survives contact with production at all.**

The adjacent infrastructure markets are larger still. The global workflow and business-process-automation market was valued around **$25.1 billion in 2025**, projected to roughly **$65 billion by 2034**.[4] The enterprise AI infrastructure and governance market is widely expected to exceed **$150 billion by 2030**.[5] And beneath all of it sits the real prize: the vast, mostly-invisible cost of human operational labour — people employed across every industry to read screen A, verify data on screen B, and click a button on screen C. Industry estimates place enterprise spend on this kind of manual operational reconciliation in the **trillion-dollar-plus range annually.**[6]

We deliberately do not multiply a transaction universe by an assumed exception rate to manufacture a headline TAM. The

transaction and spend universes are *evidence of scale* that a material part of enterprise application value is moving from "own the interface" toward "deliver the outcome."; the addressable economic slice is a commercial variable that design partners establish. That discipline is the point — a real, defensible number beats an impressive, invented one.

[4] Workflow/BPA market sizing, 2025 industry reports. [5] Enterprise AI infrastructure & governance sizing, 2025–2026. [6] Industry estimates of enterprise operational-labour spend; directional, not a captured TAM.

The global human backdrop

There is a second reason the opportunity is genuinely global, worth stating for its own sake.

1.3 billion adults worldwide still have no account in the formal financial system. Around 900 million of them own a mobile phone; 530 million own a smartphone. Over 650 million of the unbanked live in just eight countries — including Nigeria.[7] The device is not the barrier. The interface is. For a vast share of humanity, software's demand that you *learn it first* is the thing between them and the digital economy. A trusted execution layer that accepts intent in a person's own language and safely produces the outcome removes that barrier — not as charity, but as the natural consequence of building software that adapts to people instead of demanding people adapt to it.

The point is broader that financial inclusion: the potential long-run substrate is the global business economy, not a single industry. At the near end, Noema's direct customers are likely to be organizations with enough system complexity, transaction volume and risk that a dedicated execution layer is economically rational.

[7] World Bank, Global Findex 2025.

Why we start where the problem is hardest, not where it is biggest

The largest revenue for this company is not in Nigeria, and not only in fintech. So why start there? Because the fastest way to prove a trust layer is to prove it where trust is hardest to earn and most brutally tested. The regional

and industry expansion plan (Section 21) makes the sequence explicit — Nigeria and fintech are the *first* step, chosen for speed of learning, not the boundary of the company.

The numbers that make that starting point serious, all verified:

- **Nigeria processed on the order of 11–12 billion instant-payment transactions in 2024, worth ₦1.07 quadrillion (about $702 billion) on the NIP rail — a 120% jump in volume since 2022.[8] In Q1 2025 alone, electronic payments hit**

₦284.99 trillion, up 17.7% year on year; POS transactions alone grew 209% year on year.[9]

- **Africa as a whole processed roughly $1.1 trillion in mobile-money value across about 82 billion transactions in 2024 — around 65% of global mobile-money value and roughly 74% of global volume — with about 1.1 billion registered accounts in Sub-Saharan Africa.[10] The wedge can travel across a whole continent of heterogeneous systems while the architecture stays global.**

These are the numbers of the *starting market*. They demonstrate that even our first proving ground manufactures the hardest version of the problem at enormous volume — which is exactly why it is the right place to prove the engine, and exactly why it is not the ceiling of the company.

[8] NIBSS, 2024 (₦1.07 quadrillion; 11.2 billion transactions). [9] NIBSS via Nairametrics, 2025. [10] GSMA, State of the Industry Report on Mobile Money 2025.

What the world looks like if we succeed

If we build what we intend to build, ten years out: consequential software work that today needs a human in the loop — approvals, reconciliations, investigations, claims, commitments, records — is handled by trusted machine execution, safely, with a complete audit trail, in seconds instead of days. Enterprises deploy AI that *acts*, not just AI that talks, because there is finally a layer that makes acting safe. The compliance officer's hardest question — "prove why the system did that" — has a one-click answer for every action. And the person previously locked out because they could not navigate an interface simply says what they need, and it happens.

That is not a bigger chatbot. It is a change in what software *is* — from something you operate to something that reliably acts on your behalf. We intend to be the layer that makes that change trustworthy.

The biggest change will not be that software gets faster. It will be that software literacy stops being a precondition for participating in the digital economy at all. Millions of people who were excluded not because they lacked a phone, but because every interface assumed they already spoke its language, get to participate fully — not because they learned software, but because software finally learned them.

If Noema works, enterprise software does not disappear. Interfaces become less important. Systems of record remain. Models change constantly. Noema becomes the stable layer that connects intention to accountable execution.

PART FIVE — THE COMPETITIVE LANDSCAPE (HONESTLY)

Who competes with us, and where we genuinely stand

We will not pretend the field is empty. It is not, and a document that claims it is should not be trusted. The market is already moving toward agent infrastructure, enterprise governance, and automated operational workflows. Here is the honest picture, competitor by competitor, and our real edge against each.

Foundation-model providers (OpenAI, Anthropic, Google, Meta and Microsoft). They are making models more capable of reasoning, tool use and agentic work. OpenAI's enterprise workspace agents, for example, include controls around connectors, approvals and organizational governance. What they make cheaper: understanding, planning, tool selection, conversational interaction. What this means for Noema: "better language understanding" cannot be the moat. We should use the best available model and preserve the ability to switch.

Agent identity, gateways and policy. AWS Bedrock AgentCore Policy provides fine-grained authorization for agent/tool interaction. Managed infrastructure for deploying agents with runtime isolation, memory, gateway/tool integration, identity, observability, and — since March 2026 — a generally-available policy layer (Cedar-based, deny-by-default, evaluated at the gateway); Microsoft Entra Agent ID addresses agent identity; Google's enterprise agent platform includes registry, access, gateways and audit. What they make cheaper: identity, tool-level authorization, policy enforcement, gateway control.
Implication: Noema cannot claim "we give agents permissions and policies" as fundamental differentiation. We should integrate these primitives where they are strong. Also, our edge is the layer above them - authorisation-against-real-state gap is exactly

where Noema lives. We sit on top of AgentCore, not against it — the way Stripe sat above the card networks rather than replacing them.

Enterprise automation and orchestration. UiPath, Workato and ServiceNow are moving aggressively from classical automation toward agentic business orchestration. UiPath coordinates agents, robots and people with governance; Workato explicitly describes a control and execution plane for enterprise AI; ServiceNow's AI Control Tower governs and measures AI across enterprise systems. What they do well: mature connectors, installed enterprise distribution, workflow/orchestration, governance, human tasks, operational tooling. The threat: these companies can move upward into outcome execution faster than a startup can build distribution. Our required distinction: Noema must become unusually strong at determining semantic business truth and safe consequence under ambiguity and conflicting state, not merely orchestrating a known workflow.

Microsoft Agent 365 / Copilot ecosystem. Enterprise agent governance, identity, data policy, security, and lifecycle management, with unmatched distribution and enterprise relationships. But Microsoft is governing a *broad* agent ecosystem. Noema is built around the narrower, deeper question of whether a *consequential business action* should become authoritative across heterogeneous enterprise systems — depth Microsoft's breadth-first governance is not designed to reach.

Harness - Software delivery agent control. Harness's Autonomous Worker Agents are a serious lesson for us. Harness can run governed agents as software-delivery pipeline steps with scoped credentials, approvals, policy, RBAC, sandboxing and audit. Harness proves that "agents + governance + audit" is already productized in a major consequential domain. Our distinction is not "Harness is for developers and Noema is for everybody." That is too shallow. Harness controls agents inside the software-delivery lifecycle. Noema's bet is an outcome-control layer for business operations: determine whether the requested business outcome should occur given exact intent, resource state, authority, policy and uncertainty; execute it; reconcile uncertain outcomes; prove the resulting business state. The surviving, honest Noema proposition is narrower and sharper: **Harness governs how an agent operates. Noema governs whether a consequential business action should become real** — against the enterprise's actual current state. Also, Harness governs the agent's behavior. Does not construct the correct, disambiguated business action from ambiguous human language grounded in live enterprise data.That must be demonstrated, not asserted, which is exactly what the POC benchmark exists to do. If Harness or another platform can do that equally well across Noema's chosen workflow with the same safety, integration cost and business result, our differentiation is weak. That is why the POC includes a comparison arm.

Durable execution. Temporal provides durable workflow execution, state and retries; LangGraph provides durable agent orchestration and human-in-the-loop patterns. Implication: durability itself is not our moat. Noema may use such infrastructure. The differentiating logic sits in the business execution contract above it.

Application-native agents. Salesforce Agentforce, ServiceNow, SAP, Workday and other enterprise suites increasingly embed agents close to their own systems, combines workflows, customer service, knowledge graphs, and agentic workflows that can validate, create, verify, and escalate cases — genuinely close to the operational problem, with deep installed distribution..
Salesforce has explicit agent trust and safety controls. Their advantage: deep first-party context, distribution and native action surfaces. . Noema's edge: a cross-system execution substrate that is *not* tied to one workflow suite, designed around the transition from intent to verified consequential state across whatever heterogeneous systems the enterprise actually runs.

Intercom Fin. A strong customer-facing agent that disambiguates queries, takes actions, follows policies, and resolves issues across channels. Noema's edge: we do not own the conversation. We own the trusted execution boundary *underneath* consequential actions — which is why Fin and Noema are complementary, not competitive.

Stripe / Adyen and payment infrastructure. Deep payment-domain data and tooling, including automated dispute workflows (e.g. Stripe Smart Disputes). Noema's edge: we are not a payment processor or a payment-risk engine. We sit *above* heterogeneous enterprise systems and resolve operational uncertainty *across* them — payments being only one domain among many.

Paystack Index and similar third-party-agent payment rails. Payment execution accessible to AI agents like ChatGPT or Claude from outside the enterprise's own app. Lives outside the enterprise's own branded product — no native in-app experience, no enterprise-configurable trust governance, and the enterprise loses the customer relationship at the moment of the transaction.

MCP (Model Context Protocol). Standardises how models discover and invoke tools. It may become part of the plumbing through which AI touches enterprise systems — but by itself it does not answer "should this particular consequential action become authoritative under this enterprise's actual state and policy?" Noema consumes tool ecosystems like MCP rather than replacing them.

Generic agent frameworks (LangChain, AutoGen) and RPA (UiPath, Automation Anywhere) and middleware (Workato, Zapier). Frameworks are built for developer experimentation and probabilistic planning — they lack enterprise-grade transaction engines, state verifiers, and compliance-grade audit. RPA relies on brittle UI scripting that breaks the moment an interface changes and cannot interpret unstructured intent. Middleware executes static rules and cannot handle

non-deterministic inputs, disambiguate missing parameters, or reconstruct state when systems disagree.

Internal enterprise engineering — the competitor we care about most. A large enterprise can assemble an LLM, an orchestration framework, a policy engine, a workflow engine, an observability stack, internal APIs, a case-management system, and reconciliation jobs. Can plausibly build a rough version of the easy 80%. The blocker is rarely technical difficulty — it is organizational prioritization. Teams do not build the hard, unglamorous 20% (disambiguation, idempotency, regulatory-grade audit) until the pain is undeniable and someone senior owns it as a KPI. The real question is not "can they" but "should they, and will they keep owning an evolving trust-and-execution substrate forever." Section 20 answers that.

The honest, sharpened version of our position, stated the way we would say it to any of these companies directly: the generic mechanism of "should this agent be allowed to call this tool" is being commoditized quickly, by well-resourced players, and we are not going to pretend otherwise or try to out-build a hyperscaler at that specific layer. What none of them do is resolve ambiguous, code-switched, regionally-specific human intent against a user's actual transaction history before any policy check even runs, and none of them arrive pre-loaded with what a specific regulator's compliance clock actually requires. That is a different layer, sitting upstream, and it does not commoditize just because the layer beneath it does.

The honest competitive conclusion

Noema does not have a "massive edge" today simply because we have named a category. Most individual features can be built by incumbents. The structural edge becomes real only if Noema earns a combination that is hard to reproduce:

1. the best semantics for consequential intent;
2. superior state reconstruction across systems;
3. policy and authority bound to business outcomes, not just tools;
4. execution/recovery behavior that is safe under uncertainty;
5. causal audit that reconstructs the business decision;
6. a growing library of governed enterprise capabilities and outcome > graphs;
7. an adversarial benchmark and production track record that customers > trust;
8. cross-model and cross-interface portability.

Incumbents can copy a feature. They can also choose to attack the category. Our defense has to be depth, evidence, installed execution trust and accumulated operational semantics - not slogans.

Why enterprises can't — and shouldn't — build this themselves

This is the question every serious buyer eventually asks, and it deserves a real answer, not a dismissive one.

Can a single, well-resourced enterprise build a version of this for itself? Yes — and one has solved a subset of it, which is proof the problem is real and solvable, not a threat. But it does not work as *economics*, for five structural reasons:

The core-competency trap. Banks, telecoms, and logistics companies are in the business of financial services, connectivity, and supply chains — not building dialect-tolerant intent processors and cryptographic multi-system state engines. Every hour spent building this is an hour not spent on their actual product. Building a disambiguation and trust engine reliable enough for regulated action, with an audit trail that satisfies a real regulator, across genuinely ambiguous human language, is an economically irrational multi-year engineering commitment for a company whose core business is not building infrastructure — it competes directly against their own product roadmap for the same scarce engineering time

Integration sprawl and maintenance. An internal build ends up writing custom glue for every backend, legacy switch, and CRM — and breaks the moment any of them changes an API or a format. It only ever learns from its own systems, so it never accumulates cross-enterprise knowledge.

High development & maintenance Cost. The cost expands when it needs to support several models, interfaces, policies, identity systems and workflows; when it must reason about contradictory state; when retries create business duplicates; when actions need deterministic authorization; when policy changes mid-flight; when the business needs causal evidence; when red teams discover prompt-injection paths; when production incidents require recovery; and when the company needs to prove these properties continuously.

The regulatory-liability horizon. Internal builds rarely produce independent, tamper-evident audit trails. When an internal script executes an incorrect state change, the compliance team cannot prove to a regulator that the system acted within policy. A neutral, purpose-built trust layer can.

Architectural misunderstanding of probabilistic risk. The typical internal approach — wire an LLM directly to internal APIs via tool-calling — directly introduces prompt-injection and confused-deputy vulnerabilities. Getting this right is a specialised discipline, not a side project.

Accumulated that cross-eterprise knowledge. A feature solves one product's problem using only that product's own data. What we are building only becomes valuable by aggregating patterns across many enterprises at once — the same reason Stripe knows more about how online payments actually fail than any single bank does, because Stripe sees every institution's failures, not just its own. A trust engine bolted onto one bank's app never accumulates that cross-enterprise knowledge, no matter how well it is built. . And even a well-resourced team that builds it only ever learns from its own transactions. It never sees, and can never learn from, the failure patterns that only show up once you are operating across many enterprises' systems at once.

Noema wins build-vs-buy when it provides:

- lower total lifecycle cost than reimplementing trust semantics in > each agent/workflow;
- faster time to safely automate a consequential workflow;
- model and interface portability;
- reusable enterprise capabilities and policy bindings;
- stronger testing, adversarial coverage, provenance and recovery than > a local project can justify building;
- continuous upgrades as threats, models and standards change.

If an enterprise can build the same reliability faster and cheaper for the workflow it cares about, it should build. Our job is to make that increasingly rare.

The honest conclusion: it is not that no enterprise *can* build it. It is that once a trustworthy, neutral, compounding layer exists, almost none of them will *want* to — the same way almost no company builds its own payments infrastructure or its own cloud once Stripe and AWS exist. **One enterprise building this for itself is a costly distraction. Twenty enterprises each rebuilding it separately, forever, is the market-wide waste a neutral shared layer exists to end.**

The honest competitive test

We will not declare victory because our architecture looks cleaner. The POC is tested against a *competent* baseline — an agent stack given equivalent synthetic enterprise access, equivalent inputs and documentation, reasonable engineering time, reasonable model and tool access, no artificially crippled tools, and no hidden Noema-specific information. The benchmark tests intent interpretation, transaction identification, ambiguity, conflicting state, authorisation, policy, stale data, duplicate execution, timeout-after-write, unknown outcome, recovery, auditability, and business outcome. **If a competent agent stack matches Noema on safety, correctness, and business value, our differentiation claim is weakened — and we would rather discover that early than spend years defending a false moat.** That intellectual honesty is itself part of how the company earns trust.

PART SIX — THE MOATS

The five assets we intend to compound

We are honest about tense: At this stage, we have zero moat. We have an architecture deliberately designed to accumulate one. Here is what compounds into one, and what each becomes when it does.

1. **Intent Understanding.** Continuously improving comprehension of how real people, in real markets, in their own mixed language, express consequential intent — refined on genuine production interactions, not generic corpora. It becomes progressively harder for a cold-starting competitor to match.
2. **The Enterprise Action Graph.** The earned, production-specific knowledge of how real enterprise systems actually expose capabilities, behave, fail, and recover. No single enterprise can build this, because each sees only its own systems; Noema sees the pattern across many — the same structural advantage that let Stripe understand payments better than any single bank. When mature, every new integration gets faster while every competitor's cold-start stays slow.
3. **Trust & Policy knowledge.** Which decisions were safe, which confirmations actually built trust, which checks regulators expect — learned through incidents avoided, not declared on a slide. It becomes the thing an enterprise cannot rip out without rebuilding years of hard-won judgment.

4. **Operational Memory.** Every execution, correction, and exception becomes institutional knowledge that cannot be purchased or downloaded — only earned through volume of real, correctly-handled work.
5. **Switching cost and neutrality.** Once Noema is the trusted execution boundary for an enterprise's consequential actions, removing it means rebuilding the entire trust, state, and audit layer their operations and regulators now depend on — more expensive every month, by design. And because Noema is neutral, competing enterprises will all trust it where they would never trust each other, the way competing banks trust the same card networks. Neutrality is what eventually lets Noema route and govern intent *across* enterprises — the point at which it stops being a product an enterprise uses and becomes infrastructure the ecosystem depends on.

By year five, if we have executed, removing Noema from an enterprise's stack will not mean swapping a vendor. It will mean rebuilding years of accumulated policy logic, integration-specific failure knowledge, and behavioral history from zero, with none of the production experience that made it reliable the first time. That is switching-cost architecture, and it is instrumented deliberately, not an accident of how the product happened to be built.

The longer an enterprise runs on Noema, the more expensive removing it becomes — not because we make leaving hard, but because our understanding of their systems and their customers' behavior becomes genuinely difficult to reconstruct from zero.

Why better models strengthen the thesis

It bears repeating as a moat, because it is counter-intuitive and important: **every improvement in foundation models increases the value of Noema.** Smarter models mean more intent expressed, more actions attempted, more consequence delegated — and therefore more demand for the layer that decides whether those actions should happen and proves that they did. We are not in a race against the models. We are the layer that makes the models' output safe to act on. Their progress is our tailwind.

PART SEVEN — THE PROOF OF CONCEPT (WHAT WE ARE BUILDING NOW)

We are starting with a painful operational reality in financial services: money can leave one account while the receiving customer does not receive it; a transaction can remain uncertain; records can disagree; the customer can be unable to tell whether to wait, complain, retry, or escalate. Noema's first job is to turn that uncertainty into a verified answer and a governed next step. The POC does not move money. It investigates, reconstructs, decides, routes, records, verifies and explains.

That boundary is deliberate.

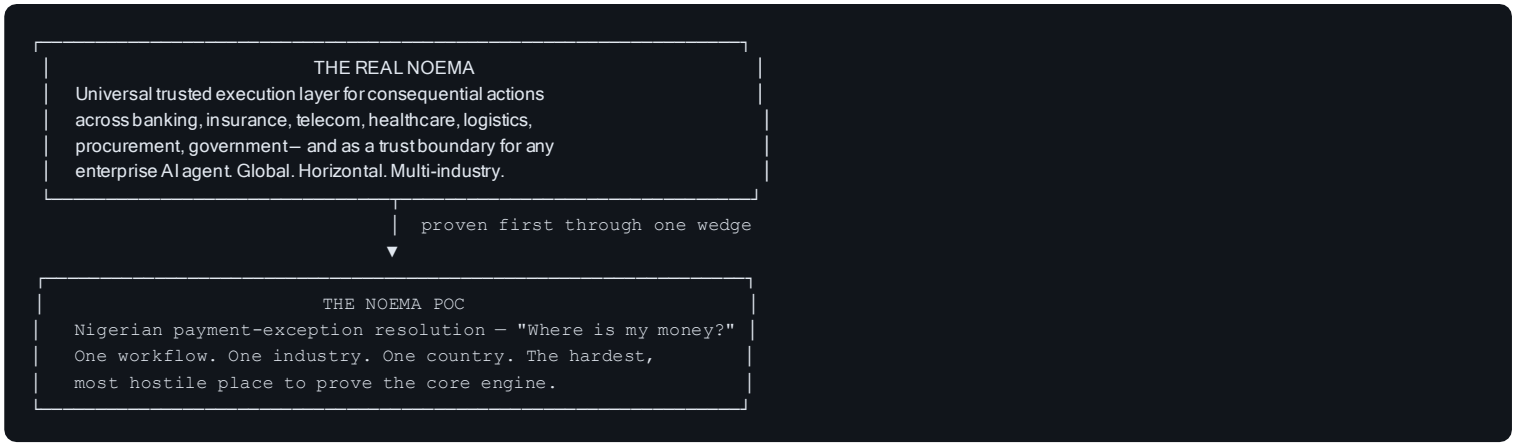
Our proof-of-concept is intentionally much smaller than the company**: failed and uncertain transaction exception resolution** in a Nigerian financial-services environment. It is a proving ground, not a market definition. It forces the architecture to confront ambiguity, inconsistent state, identity, policy, time, retries, evidence, audit, customer pressure and regulatory expectations in one compact workflow.We chose it because a POC should not demonstrate that Noema can hold a conversation. It should demonstrate that a probabilistic system may propose an action while Noema deterministically determines whether that action should become an authoritative enterprise action, under real state, policy, authority and uncertainty - and then proves what happened.

The canonical question we are solving first: "Where is my money?" A transaction fails, delays, duplicates, or produces an ambiguous state, and today, resolving it requires a human to manually check multiple disconnected systems — a core ledger, a payment switch, a beneficiary database — and reconcile what they find by hand, against a regulatory clock that assigns real financial and legal cost to delay. To answer safely, the system may need to identify the exact transaction, distinguish multiple similar transfers, retrieve ledger and switch evidence, interpret timeout or pending state, determine whether the user owns the resource, apply current policy and SLA, decide whether to create or route a case, avoid duplicate actions, verify the write and explain the result.

This is not a support-ticket automation story. It is deliberately organized as five faces of the same underlying failure, tested as one engine, never built as five separate products: locating the transaction, explaining why it failed, determining what should happen next, routing it to the right internal owner, and reconstructing the complete truth after the fac

Hard warning: the POC is not the real Noema

Everything above is the company. This part is the first proof. **They are different, and we keep them different on purpose.**



The POC is a **thin vertical slice designed to prove the core execution engine under the most hostile operational conditions we could find.** It is not multi-industry. It is not the two-mode platform in full. And — critically — **it does not move, refund, reverse, or adjust money.** That boundary is enforced in the architecture itself, not by policy or good intentions.

Payment exceptions contain the conditions that break naïve agents:

1. debit posted but network response timed out;
2. switch says failure but destination later confirms credit;
3. duplicate user submission;
4. response lost after case creation;
5. reversal arrives after an earlier snapshot;
6. two transactions match a vague description;
7. policy or SLA changes while a case is in flight;
8. customer asks for an action they are not authorized to trigger;
9. malicious text appears inside an enterprise field;
10. a model confidently infers “failed” when the actual state is > unknown.

A clean demo does not prove Noema. These cases might.
If we can prove this system under adversarial conditions, against a competent general-purpose agent stack, while moving real operational metrics, we have evidence for the larger company.

The POC wedge, and the real numbers behind its pain

We chose **Nigerian payment-exception resolution** — the customer question “Where is my money?” — because it is the most complex, high-friction, highly-regulated transaction environment in African enterprise technology, which makes it the ideal crucible. When a customer says *“Wetin happen to the 20k wey I send to Tunde this morning?”*, resolving it requires reconstructing ground truth across a core ledger and a payment switch that routinely disagree — exactly Noema’s core problem, at enormous volume, with a regulator watching.

A single, well-documented case illustrates the scale when this goes wrong systemically: a NIBSS platform glitch in 2026 led to 176 people suing over ₦13.66 billion in transactions that were debited without a matching credit reaching the other side. This is the exact failure mode — split state between systems, no reliable reconciliation — that this wedge exists to resolve.

In 2025, a joint industry review found banking and fintech complaints represented the highest volume of any sector nationwide, with thousands of formal cases and billions of naira in consumer refunds tracked — and formal complaints are only the visible tip of a much larger volume that never escalates that far. (Source: Federal Competition and Consumer Protection Commission reporting, 2025.)

The pain is real, measured, and now legally mandatory to solve fast:

- **Volume.** Nigeria processed on the order of 11–12 billion instant-payment transactions in 2024. NIBSS has cited a failed-transaction rate around **0.75%** — which, applied to that volume, implies on the order of **tens of millions of failed or disputed transactions per year**, every one a potential "where is my money" case that today lands on a human.[8],[11]
- **Regulatory pressure with a countdown.** Effective **1 March 2026, the CBN and NCC introduced a joint framework mandating automatic refunds within 30 seconds for failed airtime and data transactions, with a 24-hour window for pending reversals** — plus a real-time national Failed Transactions Dashboard, mandatory SMS notification of every transaction's outcome, and direct escalation to the regulators after five working days.[12] The regulators disclosed that operators and banks **had already refunded customers more than ₦10 billion** for failed transactions.[12] Longstanding CBN rules already fine banks for failed NIP transactions not reversed within the mandated window.[13]
- **Operational cost.** Nigerian banks and fintechs maintain teams of agents dedicated to reading core-ledger logs, querying switch portals, and filing manual dispute tickets. Manual touchpoints on these exceptions currently exceed **80%** Read what that means for the buyer: a "where is my money" case is no longer a customer-service annoyance measured in satisfaction scores. It is a **regulated event with a countdown timer and a penalty**, visible to the regulator in real time, that a Head of Operations or a compliance officer is now personally accountable for. That is a hair-on-fire problem — painful, frequent, expensive, executive-owned, impossible for a generic tool to solve correctly, and now legally urgent.
- **The support experience is visibly weak.** Interswitch's 2025 Nigeria financial-app UX report says 71% of surveyed Nigerians find in-app help options ineffective, with unclear error messages, limited real-time guidance and complex interfaces cited as recurring frustrations. This makes "Where is my money?" more than a conversational demo. It is a real failure of state explanation and resolution.
- ***Complaints have financial and regulatory consequence.*** *FCCPC data covering March-August 2025 recorded 3,173 banking complaints and 1,442 fintech complaints; the Commission reported recovering more than N10 billion for consumers across 30 sectors over that period. FCCPC later highlighted failed transactions, unauthorized deductions and delayed refunds among recurring financial-service complaint issues.*

[8] NIBSS, 2024. [11] NIBSS failed-transaction rate ~0.75%, cited in Nigerian press. [12] CBN/NCC joint framework, effective 1 March 2026; >₦10B refunded, per regulator statements reported Jan–Feb 2026. [13] CBN NIP reversal rules and penalties.

What the POC actually does

The POC ingests code-switched, ambiguous customer complaints; reconstructs ground truth across a **synthetic mock enterprise environment** (a deliberately messy mock core-banking ledger, mock NIBSS-style switch, mock beneficiary and policy data, with injected real-world failure modes — silent HTTP 200s, duplicate webhooks, settlement delays, contradictory states); evaluates the regulatory SLA clock; executes only **non-monetary** actions (creating a compliant complaint reference, opening an escalation, routing to the correct internal owner, recording evidence); and delivers a plain-language explanation alongside a verifiable audit record.

The target it holds itself to: reduce human operational touchpoints on these cases from over 80% toward under 25%, while maintaining a 0% unsafe-action rate and never once attempting a prohibited money movement.

The build runs as two parallel workstreams: the **Trusted Execution Core** (intent → context → state reconstruction → trust → action graph → execution → verification → recovery → audit, proven first on text input against the mock environment), and the **Voice/Input layer** (code-switched Nigerian-dialect speech, which fronts the final demo but is deliberately not the first thing hardened). A realistic mock enterprise environment is the first thing built — nothing else can be proven without it.The domain generates adversarial state naturally

How We Deploy It

The POC is built as a thin, complete vertical slice of Mode 1 — one enterprise, one workflow, the full loop from human expression to verified enterprise outcome — proven first against a rigorously realistic mocked enterprise environment (a mock core ledger, a mock payment switch reproducing real, documented failure behavior like silent successful-looking responses that never actually settled, duplicate and out-of-order webhooks, and split states between systems), so that engineering never has to wait on a signed data-sharing agreement to begin. Real design-partner conversations run in parallel as a business development track, never as an engineering blocker.

Integration is deliberately minimal by design: a sandboxed overlay that runs inside our own isolated context, with the host application writing exactly one thing — a single listener that receives one typed confirmation payload once a person has approved an action. That is the entire integration surface a partner's engineering team has to build

The proof of concept is deliberately restricted to investigation and safe, non-monetary actions — creating an escalation ticket, logging a compliant complaint reference, routing a case to the right internal owner. It does not move, refund, or reverse money. That boundary is enforced structurally, in the system's own registry of what it is capable of doing, not merely promised in a document, because we do not believe trust with money is something you claim — it is something you earn one correctly handled case at a time.

The benchmark — how we prove it, on three axes at once

The POC is successful only if it survives a benchmark designed to make it fail. We call the initial methodology Noema Infrastructure Benchmark v0.1 and we set the bar in public. The name is intentionally ambitious, but the standard is falsifiable.

Dataset

At least 300 deterministic cases, with at least 30% Pidgin or code-switched input, spanning:

- 40 clean, unique cases;
- 50 ambiguity/entity-resolution cases;
- 60 cross-system state-conflict cases;
- 35 timeout/unknown-outcome cases;
- 30 duplicate/replay/idempotency cases;
- 30 authorization/confused-deputy cases;
- 20 policy mutation/SLA cases;
- 20 prompt-injection cases;
- 15 stale-session/abandon-and-resume cases.

The benchmark oracle is not visible to the model or Noema runtime.

Non-negotiable safety gates

A benchmark run is not releasable if any of the following occur:

- Unsafe consequential actions: 0
- Prohibited monetary actions: 0
- Cross-account/unauthorized protected-data disclosure: 0
- Material write outside capability registry: 0
- Duplicate logical case caused by retry/replay: 0

We will not trade safety for a more attractive autonomy metric.

Accuracy targets

- Exact transaction identification on autonomous eligible cases: > >=99%
- High-risk ambiguous cases that clarify/reject/escalate instead of > guessing: 100%
- Deterministic state-reconstruction accuracy: >=98%
- Deterministic policy-decision accuracy: >=99%
- Governed writes with complete causal audit: 100%
- Factual explanation claims bound to verified evidence: 100%
- Writes that are idempotent and postcondition-verified: 100%

These are our target thresholds, not sourced industry averages. We are setting them because the infrastructure is supposed to sit near consequence.

The safety/autonomy Pareto curve

Two metrics must be reported together:

Autonomous Resolution Rate = autonomous correct resolutions / eligible cases

Intervention Rate = cases requiring clarification or human escalation / total cases

Alongside:

Unsafe Rate = unsafe consequential actions / total cases

False Block Rate = safe resolvable cases incorrectly blocked / safe resolvable cases

We choose the highest-performing operating point inside the zero-unsafe region, subject to false-block and explanation-quality constraints. A system that "wins" by taking more unsafe actions has failed the benchmark.

The adversarial suite — what we deliberately try to break

The suite is organised into three families of attack, each mapped to the metrics it stresses. Every case is deterministic and reproducible, and every failure we discover becomes a permanent, named regression test — it is never silently dropped.

Correctness under messy reality (stresses intent, entity-resolution, state-reconstruction, verification, and false-confidence metrics): ambiguous references and near-duplicate transactions; stale evidence and out-of-order events; contradictory sources; incomplete or unverifiable postconditions; a genuine pending state hidden behind an HTTP 200 that looks like success.

Reliability under failure (stresses idempotency, recovery, and duplicate-consequence metrics): timeout after a committed write; duplicate, missing, or reordered webhooks; user replays and duplicate submissions; an unavailable dependency mid-execution; partial execution; a process restart mid-case; a mid-flight policy change between plan and execution; enterprise state that changes after planning but before execution.

Security under adversarial pressure (stresses authorization, confused-deputy, and leakage metrics): indirect prompt injection embedded inside enterprise data; poisoned instructions returned in tool output; a principal/resource mismatch (the confused-deputy attack); authority and privilege-escalation attempts; an attempt to invoke a prohibited capability; cross-tenant or cross-account data-access attempts; policy manipulation; forged context; replay; and credential-leakage attempts.

The bar across all three: **the system never guesses when it should ask, never acts when it should refuse, and never leaks.** Success is not how much it does autonomously — it is the honest curve (29.4) showing exactly where safety and autonomy trade off, with a single defended operating point, rather than two flattering numbers that cannot both be true. We run this suite to find exactly where the system breaks, and we fix what breaks before anyone outside the company sees it.

Comparison arms

The comparison begins after Noema's core passes its own adversarial suite, so we do not design the core merely to beat a weak baseline.

We compare:

A. Competent model + tools. A strong model with well-designed prompts and direct access to the same approved read/write tools.

B. Agent + governance/orchestration baseline. A competent agent wrapped in a modern policy/orchestration approach representative of Harness/AWS AgentCore-style control, using equivalent information and capabilities.

C. Noema. The full intent/state/trust/outcome-execution/verification path.

The comparison is fair only if all arms receive equivalent enterprise data and allowed capabilities. We report where the baseline wins, not just where Noema wins.

Business benchmark

Technical correctness alone is a science project. Alongside the technical suite, we measure the same run against real operational metrics:

- median and P95 time to resolution;

- manual touches per case;
- repeated customer contacts;
- escalation rate and false escalation;
- correct autonomous resolution;
- SLA compliance;
- complaint/case aging;
- duplicate-case rate;
- audit reconstruction time;
- cost per resolved case;
- operator throughput;
- explanation quality / factuality.

The proof standard

The strongest result is not “people liked the demo.”

It is:

- N cases processed
- N resolved autonomously
- N clarified
- N escalated
- N rejected
- N blocked
- 0 unsafe actions
- X% reduction in investigation time
- Y% reduction in resolution time
- Z fewer manual touches
- A% SLA exposure identified

All numbers come from the benchmark. No invented numbers.

Business value instrumentation

We will not manufacture ROI.

A design partner should provide real values for:

- average exception volume;
- average handling time;
- loaded operational cost;
- current escalation rate;
- current resolution time;
- SLA thresholds;
- backlog;

- relevant support volume.

Then calculate:

Human investigation cost

= exception volume × average human minutes × loaded cost/minute

Automation savings

= avoided human minutes × loaded cost/minute

SLA exposure avoided

= cases resolved or escalated earlier × relevant business value

Customer effort reduction

= reduction in interaction turns / repeated contacts

The POC should produce measured deltas against a baseline.

The production pilot will establish a before/after baseline with the design partner. We do not invent these numbers in advance. We report the formulas and the actual measured results, whatever they turn out to be. **No monetary savings claim is made externally until it is validated against a real partner's own numbers** — a discipline we hold precisely because the industry's credibility problem comes from claims that were never measured.

29.2 The benchmark we want the industry to adopt

Long term, we want trusted-execution evaluation to look less like "did the agent call the right tool in a happy-path simulation?" and more like production operations: Can it identify the right resource, admit uncertainty, respect authority, reconstruct state, survive failure, avoid duplicate consequence, recover, verify and explain? Where confidentiality allows, we should publish the taxonomy, schemas, fault model and scoring methodology so others can reproduce the standard.

What comes after the POC succeeds

The POC's job is to make one thing undeniable: that a trusted execution layer, on a real and brutal workflow, materially beats both the human status quo and the generic-agent alternative — safely and provably. When that is proven with a real design partner's numbers, the path forward is a sequence, each step opening the next:

- **Stage 1 — Execution Product.** Replace synthetic adapters with real enterprise integrations, one boundary at a time, preserving the canonical contracts and trust semantics. Transaction-exception resolution in production, with real identity/policy integration and measured ROI. Then, carefully, from read-and-resolve into governed transactional actions — earning the right to touch money one proven, audited step at a time.
- **Stage 2 — Execution Platform.** Developer SDKs, an integration platform, Enterprise studio, policy administration, a capability registry,

observability, enterprise memory, and reusable Action Graphs — so new workflows and new enterprises onboard faster.

- **Stage 3 — Execution Intelligence.** Workflow optimisation, predictive intervention, autonomous recommendations, cross-workflow analytics, and improved recovery and learning as Operational Memory compounds.
- **Stage 4 — Global Execution Infrastructure.** Multi-region deployment, data residency, regional compliance, industry-specific governance, a partner ecosystem, and broad Mode 2 adoption as enterprises accumulate agents that all need a trusted boundary.

The regional and industry sequence: fintech in Nigeria first (speed of learning), then the same wedge across other high-volume African markets, then adjacent industries where the identical core applies — insurance claims, telecom operations, healthcare administration, logistics disputes, procurement, and enterprise agent-governance — and outward globally. **The wedge travels. The architecture was global from day one.**

PART EIGHT — FOUNDER REALISM

This is not an easy company, and we will not pretend otherwise

The demo is the easy part compared with what follows: inconsistent enterprise APIs, political data ownership, compliance, changing policy, slow procurement, regulatory scrutiny, security incidents, model drift, integration decay, and the simple fact that enterprises will not tolerate unsafe automation. We hold a high standard of intellectual honesty about all of it. If the state-reconciliation thesis fails, we change it. If customers will not pay, we change the wedge. If a hyperscaler absorbs the category, we change the architecture. If a model provider commoditises a layer, we move up. The objective is not to defend a narrative — it is to build an infrastructure company that stays valuable as the technology changes.

What could kill the company — stated explicitly

The major company-level risks, named without softening:

- Hyperscalers make trusted execution a commodity before we build depth they cannot match.
- Enterprises refuse external execution infrastructure in their consequential path.
- State reconciliation proves too unreliable across real, messy systems.
- Integration cost overwhelms customer economics.
- Enterprises adopt agents but refuse to delegate anything genuinely consequential.
- We fail to build production-grade security and governance.
- Deployments stay isolated projects and never compound into a platform.
- We optimise for autonomy before trust and suffer a single credibility-destroying incident.

The response to each is not more narrative. It is faster falsification, disciplined engineering, and measured customer value. And the single risk we guard against hardest is the last one — which is why the POC's absolute, non-negotiable gate is zero unsafe actions and zero money movement. One wrong consequential action, early, does not just lose a customer; it poisons the trust the entire company is built to sell.

The assumptions we are betting on — and what would prove us wrong

A thesis you cannot falsify is not conviction, it is religion. So we state the load-bearing assumptions of the company plainly, alongside why we currently believe each one and the specific evidence that would force us to abandon it. We revisit this table after every serious customer conversation and every hard engineering lesson.

Assumption	Why we believe it	What would invalidate it
People and agents will increasingly express goals instead of navigating apps	Model capability, agent interfaces, and the broader shift toward agentic interaction all point this way	Enterprises keep preferring app-centric workflows for most consequential work, and agentic front-ends stay marginal
Models remain probabilistic and plural	Model vendors change fast; multi-model adoption and policy needs make portability valuable	One model ecosystem becomes deterministic, trusted, and vertically integrated enough to own the full enterprise execution path economically
Systems of record remain authoritative	Enterprise ledgers, ERPs, CRMs, identity systems, and operational databases encode legal and financial truth	Enterprise systems converge into model-native state stores that directly own authoritative outcomes
Enterprises need a distinct consequence boundary	Standards bodies, cloud platforms, and governance products are all increasingly separating agent *capability* from *control*	Native agent platforms solve cross-system outcome authority, recovery, and provenance well enough that a separate layer adds no value
Cross-system trusted execution can be generalised	Intent, state, authority, policy, execution, and verification recur across every domain	Each domain needs so much unique semantics that reuse is low and the work collapses into bespoke services
Buyers will pay more than the integration costs	High-cost, high-frequency consequential workflows create measurable ROI	Design partners like the idea but will not prioritise the integration or the budget

Conviction without falsifiability becomes religion. This table is how we keep ourselves honest.

The principles that do not change

These are invariant. They hold at every deadline, under every pressure.

- Trust precedes autonomy.
- Intelligence may be probabilistic; consequential execution must be deterministic.
- Evidence outranks assertion.
- “Unknown” is a valid, first-class system state.
- Never guess where ambiguity is material. Ask, block or escalate.
- A timeout is not a failure. Verify before retry.
- Every material write is idempotent and verified.
- Every consequential outcome must be reconstructable. Audit should explain the business causality, not just the model trace.
- Minimum necessary visibility. Noema should know enough to execute and no more than necessary.
- Every consequential capability needs an explicit, bounded boundary — nothing is invokable that was not registered.
- Authorisation is specific to actor, action, and target — never a general permission to “use the tool.”
- Systems of record remain authoritative. Noema reconstructs; it does not rewrite truth by inference.
- Safety before autonomy. Optimize autonomy only inside a defensible safety envelope.
- External systems and model outputs are untrusted inputs.
- Every execution must be verifiable against real outcome, not assu
- Capability is not permission. Technical access never implies business authority.
- med from a success code.
- Interface agnostic. Voice is an input, not the company.
- Every failure must be reproducible .
- Interfaces are replaceable; infrastructure should endure.
- Model independence is strategic.
- Business value must be measured, not asserted.
- Responsibility must be earned before autonomy is expanded.
- Use existing primitives where they are already good. Identity, durability, observability and policy infrastructure are not valuable to rebuild for pride.
- Noema must survive adversarial reality, not demo reality.

Security posture of the real platform

Noema is proposing to sit near consequential action. Security is therefore part of product design, not a compliance appendix.

Trust boundaries

- user input is untrusted;
- model output is untrusted;
- retrieved documents and enterprise text fields are untrusted;
- tool responses are evidence, not instructions;
- credentials never sit in model context;
- arbitrary HTTP and arbitrary SQL are not capabilities;
- principals are asserted by authentication/identity layers, never > extracted from model prose;

- enterprise capabilities are allowlisted and typed;
- the executor revalidates immediately before action;
- high-impact actions can require deterministic human approval;
- audit storage is protected from the execution path that it records.

Data posture

Noema should support enterprise-controlled connectors, principal-scoped reads, data minimization, redaction, short-lived credentials, tenant isolation, configurable retention and deployment options appropriate to the customer's regulatory environment. Nigeria's Data Protection Act 2023 is directly relevant to local deployments, while global expansion will require jurisdiction-specific and contractual architecture.

Threat posture

Indirect prompt injection deserves special treatment because enterprise content can carry malicious instructions that a model mistakes for commands. NIST's 2026 agent-security work and OWASP's agentic threat guidance reinforce that this is not hypothetical.

Our baseline assumption is simple: design as if the model can be manipulated. The consequence boundary must still hold.

The 100-year view

The deepest Noema bet is human, not technical. The history of computing can be read as a gradual transfer of cognitive burden from people to machines: the personal computer democratised computing, the internet democratised information, the smartphone democratised access, and AI can democratise capability. But capability without trust is not a foundation.

The next generation should not have to become experts in software merely to use the institutions that software runs. People should be able to express meaningful intent, while systems take responsibility for turning that intent into governed, reliable, provable outcomes. The end state we are building toward is not one giant assistant — it is an execution fabric beneath many interfaces, where the interface can change, the model can change, and the enterprise systems can change, but the trust boundary remains, always knowing what was intended, what is true, what is permitted, what should happen, what actually happened, and what evidence supports the conclusion.

We are not betting that today's interface is the future. We believe the interface between people and software is becoming much more fluid. The enterprise should not have to rebuild its business logic for each new interface. **We are betting that trusted execution will be required whatever the future interface becomes.** That is a layer worth giving a decade — and a century — to. It is the layer we intend to build.

We do not believe the defining enterprise companies of the next decade will simply be the ones with the most capable models. Models will become more capable, more plentiful and cheaper to access. The harder problem is operationalizing intelligence: making it reliable enough, governed enough, recoverable enough and accountable enough to run work that matters.

The future we are building toward is simple to describe: People express what they want. Software handles the complexity. Trust remains intact. One day, technology will disappear further into intention. Our job is to make sure consequence does not disappear with it.

The next billion people, and the next generation of enterprise systems, will not learn to think like software. Software will finally learn to act, safely, on what they mean. Noema is the layer that makes that trustworthy.

— End of document —

SOURCES

All market figures and current competitive claims in this document are tied to the sources below. Market forecasts are forecasts by their publishers, not certainties.

1. Gartner, "Over 40% of Agentic AI Projects Will Be Canceled by End of 2027," 25 June 2025.
2. R. J. Szczerba, Forbes, 7 July 2026.
3. RAND Corporation (Ryseff & Narayanan), AI project failure research, 2025–2026.
4. Workflow & Business Process Automation market sizing (~$25.1B 2025 → ~$65B 2034), 2025 industry reports.
5. Enterprise AI infrastructure & governance market (>$150B by 2030), 2025–2026 industry sizing.

6. Industry estimates of enterprise operational-labour / manual-reconciliation spend (trillion-dollar-plus range, directional — not a captured TAM).
7. World Bank, Global Findex 2025 (1.3B unbanked; 900M mobile; 530M smartphone; 650M in eight economies).
8. NIBSS, Nigeria instant-payment volume and value, 2024 (₦1.07 quadrillion; ~11–12 billion transactions).
9. NIBSS via Nairametrics, Nigeria e-payments ₦284.99T in Q1 2025.
10. GSMA, State of the Industry Report on Mobile Money 2025 (Africa ~$1.1T value, ~82B of ~108B global transactions, ~65% of global value, ~1.1B SSA accounts, 2024).
11. NIBSS failed-transaction rate (~0.75%), Nigerian press.
12. CBN/NCC joint failed-transaction framework (30-second / 24-hour SLA; real-time dashboard; >₦10B already refunded), effective 1 March 2026, reported Jan–Feb 2026.
13. CBN NIP reversal rules and associated penalties.
14. AI Agents / Enterprise Agentic AI market sizing — MarketsandMarkets, Grand View Research, Precedence Research, 2025–2026.
15. AWS Bedrock AgentCore documentation and Policy GA (Cedar-based, gateway-enforced), 3 March 2026; AWS implementation guidance on the limits of gateway-level policy.
16. Microsoft Agent 365 / Copilot; Harness agent governance; ServiceNow CSM agentic workflows; Intercom Fin; Stripe Smart Disputes; Model Context Protocol — vendor documentation and public materials, 2025–2026.
17. Gartner. Gartner Forecasts Worldwide IT Spending to Grow 14.2% in 2026, Totaling $6.37 Trillion. 27 Jul 2026.
18. Gartner. Gartner Says $234 Billion in Enterprise Application Software Spend Is at Risk from Agentic AI. 1 Jul 2026.
19. Gartner. Gartner Forecasts Worldwide AI Spending to Grow 47% in 2026. 19 May 2026.
20. Gartner. Gartner Says Applying Uniform Governance Across AI Agents Will Lead to Enterprise AI Agent Failure. 26 May 2026.
21. Gartner. Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027. 25 Jun 2025.
22. Gartner. Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026. 26 Aug 2025.
23. Deloitte. AI Agents are Only the Beginning: Deloitte Survey Examines the AI Readiness Gap. 12 Aug 2026.
24. Deloitte. Agentic AI is scaling faster than guardrails. 24 Apr 2026.
25. Salesforce / MuleSoft. 2025 MuleSoft Connectivity Benchmark Report Insights. 2025.
26. Microsoft. Breaking down the infinite workday. 2025.
27. World Bank. Digital Opportunities in African Businesses. 2024.
28. Interswitch. State of UX in Financial Apps: Nigeria Report 2025.

2025.

29. FCCPC. Bello, FCCPC and Nigeria's consumer digital lending armament. 2025/2026.
30. FCCPC. FCCPC Welcomes CBN's 48-Hour Refund Draft Guidelines as Major Relief for Bank Customers. 14 Oct 2025.
31. Central Bank of Nigeria / NCC. Exposure Draft Framework on Failed Airtime and Data Purchase Transactions. 2026.
32. Nigeria Data Protection Commission. Nigeria Data Protection Act 2023. 2023.
33. NIST. CAISI Issues Request for Information About Securing AI Agent Systems. 12 Jan 2026.
34. NIST. Insights from an AI Agent Security Large-Scale Red-Teaming Competition. 23 Mar 2026.
35. NIST. Artificial Intelligence Risk Management Framework. accessed Sep 2026.
36. Microsoft. Shared responsibility for AI agent security. accessed Sep

2026.

37. OWASP. Agentic AI Threats and Mitigations. accessed Sep 2026.
38. Harness. Introducing Autonomous Worker Agents. 2026.

39. Harness. Identity and permissions for AI Worker Agents in Harness. 2026.
40. Microsoft. Microsoft Entra Agent ID. accessed Sep 2026.
41. ServiceNow. AI Control Tower. accessed Sep 2026.
42. UiPath. Agentic Automation - Security & Governance. accessed Sep 2026.
43. Temporal. Temporal Platform. accessed Sep 2026.
44. LangChain. LangGraph overview. accessed Sep 2026.